Experiment - 4

Student Name: Mohd Shahid
UID: 23BCS10258
Branch: BE-CSE
Section/Group: KRG-2A
Semester: 6th
Date of Performance: 05/02/26
Subject Name: System Design
Subject Code: 23CSH-314

Aim: To design an **scalable OTT platform** (Similar to **Netflix** or **Amazon Prime**).

Objectives:

- To identify **functional and non-functional requirements** of an OTT streaming system
- To design a **high-level architecture (HLD)** for a large-scale video streaming platform
- To understand **video streaming protocols (HLS / DASH)** and adaptive bitrate streaming
- To apply **CAP theorem trade-offs** in OTT systems
- To understand the role of **Kafka, CDN, object storage, chunking, and encoding pipelines**
- To study how **availability is prioritized over consistency** in media streaming systems

Tools:

- **Draw.io** – System Architecture & HLD Diagram
- **MySQL** – User, subscription, and payment data
- **MongoDB / NoSQL DB** – Video metadata storage
- **ElasticSearch** – Search functionality for movies and TV shows
- **Apache Kafka** – Asynchronous event streaming and video pipeline coordination
- **Redis / CDN Cache** – Caching frequently accessed content
- **Blob Storage (Amazon S3 equivalent)** – Video and media storage

Functional Requirements:

1. Clients should be able to create an account on the OTT platform.
2. After successful login, clients should be able to choose and manage subscription plans.
3. Clients should be able to search for movies or TV shows by title or name.
4. Clients should be able to stream videos in multiple resolutions (480p, 720p, 1080p, 4K).
5. Clients should receive recommendations for movies and TV shows (Optional)

Non-functional Requirements:

A. Scalability

- Target Users: 200–300 Million active users
- Total Videos: ~20,000 videos, average duration ~1 hour
- System must handle millions of concurrent streams

B. Consistency and Availability (CAP Theorem)

- Availability >>> Consistency

Reason:

- Video streaming must not stop even during partial failures
- Slight delays in metadata updates are acceptable
- Subscription and payment data require stronger consistency

Example:

- Video playback unavailable → Critical failure
- Slight delay in subscription update → Acceptable

C. Latency

- Target latency: 50–80 ms
- Video should play with zero or negligible buffering

Core Entities of the System:

- Client
- Subscription
- Video
- Video Metadata (Thumbnails, Description)
- Recommendation Engine

API Gateway:

- Handles authentication, authorization, routing, and rate limiting

User Service

- User registration, login, JWT-based authentication
- Stores user data in MySQL

Subscription & Payment Service

- Manages subscription plans and payments
- Ensures strong consistency for transactions

Search Service

- Uses ElasticSearch for fast movie/TV show search

Video Metadata Service

- Stores metadata (title, description, thumbnails) in NoSQL DB

Video Upload & Processing Pipeline

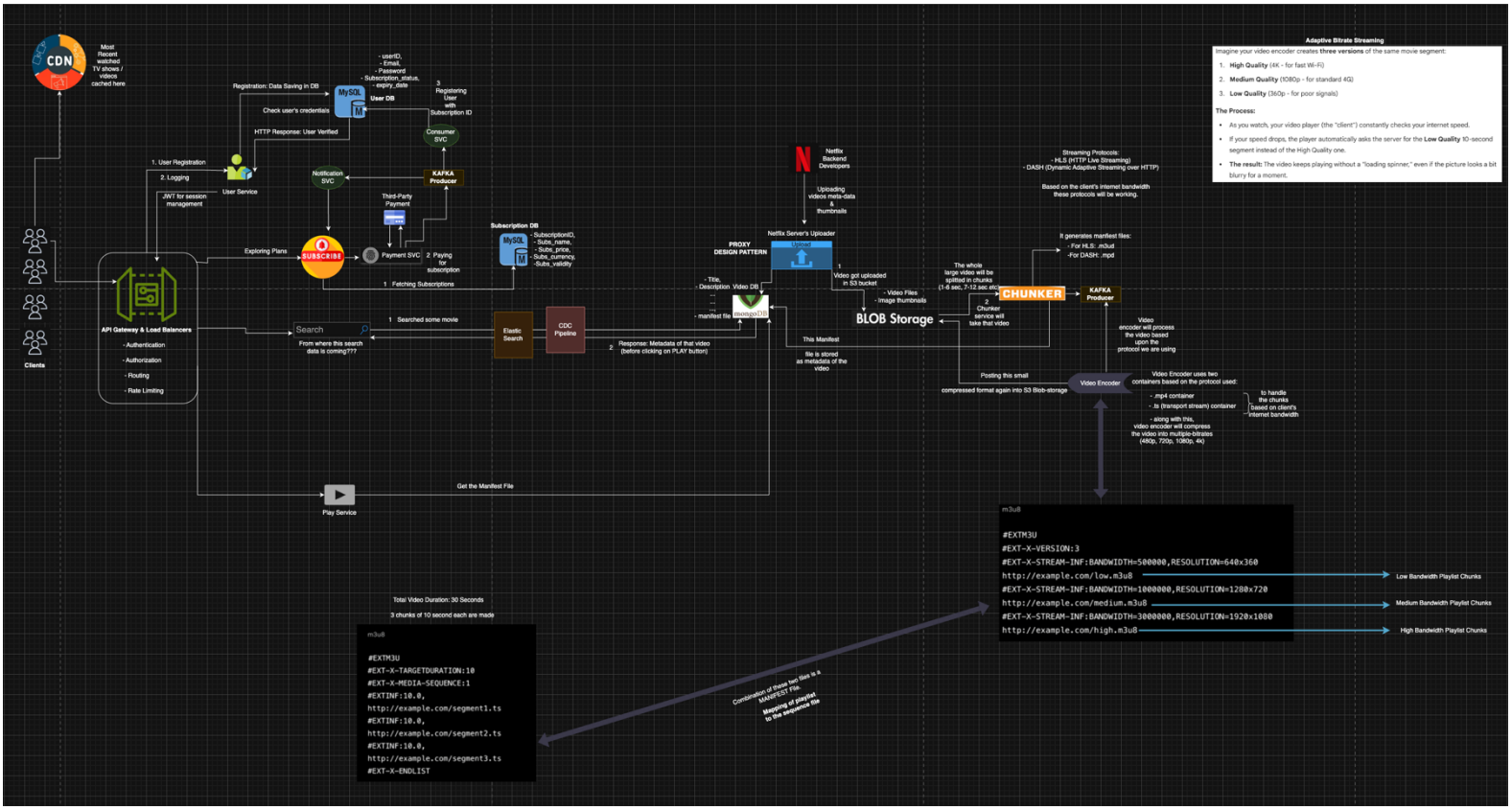
- Uploaded videos are stored in Blob Storage
- Chunker Service splits videos into small segments
- Video Encoder generates multiple resolutions (480p, 720p, 1080p, 4K)
- Kafka coordinates asynchronous processing

Streaming Service

- Generates HLS (.m3u8) or DASH (.mpd) manifests
- Client fetches chunks based on bandwidth

High Level Design (HLD):

A complete high-level design of a social media platform was successfully created, identifying its functional requirements, non-functional constraints, core entities, and feed management strategy.



Learning Outcomes:

- Understood how real-world OTT platforms like Netflix are designed
- Learned difference between functional and non-functional requirements
- Gained strong understanding of adaptive bitrate streaming (HLS/DASH)
- Understood CAP theorem trade-offs in streaming systems
- Learned how Kafka, CDN, and chunking pipelines work together